Assignment 06: 3D Ball — Level Design & Mechanics

CS 470 – Introduction to Game Dev

Due: One week from date assigned Total Marks: 20

Overview

You will extend the 3D rolling ball game built in class into a complete mini-game with multiple levels, collectibles, a HUD, and optional polish features. Start from your in-class final project (the version with camera-relative movement, jump, death zone, and victory zone).

Learning Objectives

- Design 3D levels using GridMap with increasing difficulty
- Implement scene transitions between levels
- Use Area3D signals for gameplay triggers (pickups, zones)
- Build a HUD using CanvasLayer and Control nodes
- Persist game state across scenes using an Autoload singleton

Setup

1. Start from your completed in-class project (with `ball.gd`, `death_zone.gd`, `victory_zone.gd`, and `level_01.tscn`).
2. Verify it runs: ball rolls, camera orbits, death zone reloads, victory zone prints.

Section	Marks
Part A: Level Design	6
Part B: Collectibles & HUD	8
Part C: Bonus Polish (pick any, max 6)	6
Total	**20**

Part A: Level Design [6 marks]

A-1: Second Level [3 marks]

Create `level_02.tscn` with a more challenging GridMap layout. Your level must include at least two of the following: gaps the player must jump across, narrow bridges (1-block wide), or height changes requiring platforming.

Requirements:

1. Wire the VictoryZone in `level_01` to transition to `level_02` using `change_scene_to_file()`.
2. The new level must have its own DeathZone and VictoryZone.
3. The ball must spawn at a sensible starting position.

Update `victory_zone.gd` to support scene transitions:

```
extends Area3D

@export_file("*.tscn") var next_level: String

```

```gdscript
func _on_body_entered(body: Node3D) -> void:
    if body.name == "Ball":
        if next_level != "":
            get_tree().call_deferred("change_scene_to_file", next_level)
        else:
            print("You Win!")
```

Hint

Set the `next_level` export in the Inspector for each level's VictoryZone. Level 1's victory zone points to `res://scenes/level_02.tscn`, and so on.

A-2: Third Level [3 marks]

Create `level_03.tscn` that introduces vertical platforming — blocks stacked at multiple heights requiring the player to jump upward to progress. This level should feel noticeably harder than level 2.

Requirements:

1. Level 2's VictoryZone transitions to level 3.
2. Level 3's VictoryZone has `next_level` left empty (prints "You Win!" or shows a victory screen).
3. The level should take at least 30 seconds to complete on a first attempt.

Part B: Collectibles & HUD [8 marks]

B-1: Pickup Item [3 marks]

Create a `Pickup.tscn` scene that the ball can collect.

Scene structure:

```
Pickup (Area3D)
|-- CollisionShape3D (SphereShape3D, radius ~0.5)
+-- Model (MeshInstance3D, e.g., BoxMesh or SphereMesh)
```

Requirements:

1. When the ball enters the pickup's Area3D, increment a score variable and call `queue_free()` on the pickup.
2. Add a slow rotation to make pickups visually noticeable (rotate in `_process`).
3. Place at least 3 pickups in each level (9 total across all 3 levels).

```gdscript
extends Area3D

func _process(delta: float) -> void:
    rotate_y(2.0 * delta)

func _on_body_entered(body: Node3D) -> void:
    if body.name == "Ball":
        # TODO: increment score
        queue_free()
```

Hint

To share the score across scenes, create an Autoload singleton. Go to **Project → Project Settings → Autoload**, add a new script (e.g., `GameManager.gd`) and give it a node name like `GameManager`. You can then access `GameManager.score` from any script.

B-2: HUD Score Display [2 marks]

Add a HUD that displays the player's current score.

Requirements:

1. Create a `CanvasLayer` with a `Label` node showing "Score: 0".
2. Update the label whenever a pickup is collected.
3. The score must persist when transitioning between levels (use the Autoload singleton).

Important

If you put the HUD inside each level scene, it will be destroyed on scene change. Instead, either put the HUD in your Autoload singleton (add a CanvasLayer as a child of GameManager) or re-create it in each level's `_ready()`.

B-3: Timer [3 marks]

Display elapsed time on the HUD. The timer starts when level 1 begins and stops when the player reaches the final VictoryZone.

Requirements:

1. Display time in `MM:SS` format (e.g., "01:23").
2. Timer accumulates across all 3 levels (does not reset on level change).
3. When the player wins, freeze the timer and display the final time prominently.

```gdscript
# In GameManager.gd (Autoload)
var elapsed_time: float = 0.0
var timer_running: bool = true

func _process(delta: float) -> void:
    if timer_running:
        elapsed_time += delta
```

Part C: Bonus Polish [6 marks, pick any]

Complete any combination of the following features for up to 6 bonus marks. You may attempt all of them, but the maximum for this section is 6.

C-1: Moving Platforms [2 marks]

Add a platform that oscillates between two positions (e.g., back and forth over a gap). The ball should be able to ride the platform.

Requirements:

- Use an `AnimatableBody3D` (not a `RigidBody3D`).
- Animate with a `Tween` or `AnimationPlayer`.

- The ball must move with the platform (not slide off immediately).

Hint

`AnimatableBody3D` is designed for kinematic platforms in Godot 4. It automatically transfers its velocity to riding physics bodies.

C-2: Speed Boost Zone [2 marks]

Create an Area3D that gives the ball a burst of speed when entered.

Requirements:

- Apply a one-time impulse in the ball's current movement direction.
- Add a visual indicator (colored mesh, or place it on a distinctly-colored block).
- Include a cooldown so the boost cannot be triggered repeatedly by sitting in the zone.

C-3: Respawn at Checkpoint [2 marks]

Instead of reloading the entire scene on death, respawn the ball at the most recent checkpoint.

Requirements:

- Create a `Checkpoint.tscn` (Area3D) placed at key locations in each level.
- When the ball passes through a checkpoint, store that position.
- On death, reset the ball's position and velocity to the last checkpoint instead of reloading.
- Provide visual feedback when a checkpoint is activated (e.g., color change).

C-4: Sound Effects [2 marks]

Add audio feedback for at least 3 of the following 4 events:

- Pickup collected
- Jump
- Death (falling off)
- Victory (reaching the final goal)

Use `AudioStreamPlayer` or `AudioStreamPlayer3D`. Free sound effects can be found on sites like freesound.org or kenney.nl.

C-5: Visual Polish [2 marks]

Implement any **two** of the following:

- Particle burst when collecting a pickup (`GPUParticles3D`, one-shot)
- Trail effect behind the ball (use a `MeshInstance3D` trail or particle emitter)
- Different skybox or lighting per level (modify the `Environment` resource)
- Victory animation or screen (e.g., camera zoom, confetti particles, label with final time)

Submission

1. Zip your entire Godot project folder (exclude the `.godot/` cache folder to reduce size).
2. Include a brief `README.md` (3–5 sentences) listing which bonus features you implemented.
3. Submit via the class portal before the deadline.

Important

Your project must open and run in Godot 4.6 without errors. Test by extracting your zip into a fresh folder and importing it. If it does not run, it cannot be graded.